Credit Card Fraud Detection

Using Machine Learning

Team Members

Serik Makhabbat

Sapan Merey

Kenzhebekova Sabina

Bolysbekqyzy Ayaulym

Nurzhauova Nuray

CSS 324: Introduction to Machine Learning

Contents

1 Introduction **3**

2 Dataset Collection and Preparation **3**

2.1 Dataset Head 3

2.2 Raw Data Simulation 4

2.3 Data Cleaning 4

2.4 Feature Engineering 5

2.5 Data Splitting, Scaling, and SMOTE 5

3 Research and Exploratory Data Analysis **5**

3.1 Research Review 5

3.2 Exploratory Data Analysis 6

4 Machine Learning Experiments **9**

4.1 Hyperparameter Settings 10

4.2 Evaluation Metrics 10

4.3 Model Performance Comparison 11

4.4 Confusion Matrix Analysis 11

4.5 ROC Curve Analysis 11

4.6 Precision-Recall Curve Analysis 12

4.7 Feature Importance 13

5 Final Model and Demonstration **14**

6 Analysis and Conclusion **15**

6.1 Limitations 15

7 References **15**

1. Introduction

Credit card fraud detection is an important machine learning problem because fraudulent transactions can lead to financial losses for banks and customers. Since modern payment systems process a large number of transactions every day, manual fraud detection is inefficient and difficult.

The goal of this project is to build a machine learning pipeline that classifies credit card transactions as either legitimate or fraudulent. This is a binary classification task, where class 0 represents a legitimate transaction and class 1 represents a fraudulent transaction.

A major challenge in this project is class imbalance. Fraudulent transactions are much rarer than normal transactions, so accuracy alone is not enough to evaluate the model. For this reason, this project focuses on precision, recall, F1-score, ROC-AUC, PR-AUC, and confusion matrices.

2. Dataset Collection and Preparation

The dataset contains anonymized credit card transaction records. Each row represents one transaction. The dataset includes the following columns: Time, Amount, V1 to V28, and Class.

The features V1 to V28 are anonymized numerical features created using PCA transformation. The original transaction details are hidden for privacy and security reasons. The Amount column represents the transaction amount, and the Time column represents the elapsed time between transactions.

The target variable is Class:

- Class 0: legitimate transaction
- Class 1: fraudulent transaction

2.1. Dataset Head

Table 1 shows a sample of the dataset before preprocessing.

Time Class	V1	V2	V3	V4	Amount
0.0 0	-1.3598	-0.0728	2.5363	1.3782	149.62
0.0 0	1.1919	0.2662	0.1665	0.4482	2.69
1.0 0	-1.3584	-1.3402	1.7732	0.3798	378.66
1.0 0	-0.9663	-0.1852	1.7930	-0.8633	123.50
2.0 0	-1.1582	0.8777	1.5487	0.4030	69.99

Table 1: Sample rows from the credit card fraud dataset

2.2. Raw Data Simulation

To demonstrate the effectiveness of our cleaning pipeline, we implemented a simulation of raw data inconsistencies (missing values, duplicates, and incorrect data types). This ensures that our final model is not just accurate on clean benchmarks, but is also part of a production-ready system capable of handling real-world data noise

2.3. Data Cleaning

Several preprocessing steps were applied before training the models.

- Converted Amount into numeric format
- Removed duplicate rows
- Filled missing Amount values using median imputation
- Filled missing feature values using mean imputation
- Removed rows with missing target values
- Converted the Class column into integer format

The median was used for Amount because transaction values may contain extreme outliers.

2.4. Feature Engineering

Two new features were created:

- Hour
- Log_Amount

The Hour feature was extracted from the Time variable to capture time-based fraud behavior patterns.

The Log_Amount feature was created using logarithmic transformation:

$$Log_Amount = \log(1 + Amount)$$

This transformation reduces skewness and minimizes the influence of extremely large transaction values.

2.5. Data Splitting, Scaling, and SMOTE

The dataset was split into training and testing sets using stratified sampling.

Feature scaling was applied using StandardScaler because the dataset contains numerical features with different ranges.

Since the dataset is highly imbalanced, SMOTE was applied to the training data. SMOTE generates synthetic fraud examples and helps the models learn minority class patterns more effectively.

3. Research and Exploratory Data Analysis

3.1. Research Review

Three related sources were reviewed before building the models.

The first research paper, *A Supervised Machine Learning Algorithm for Detecting and Predicting Fraud in Credit Card Transactions* by Afriyie et al., compared Logistic Regression, Decision Tree, and Random Forest for fraud detection tasks. The paper showed that ensemble methods such as Random Forest perform strongly because they can capture complex transaction patterns while reducing overfitting. Based on this paper, Random Forest was selected as one of the primary models in this project.

The second source was the Kaggle notebook *creditcard-fraud-random-forest-example*. This notebook demonstrated a practical fraud detection workflow including preprocessing, feature scaling, model training, confusion matrix analysis, and evaluation metrics. From this source, we adopted the idea of building a complete preprocessing and evaluation pipeline instead of directly training models on raw data.

The third source was the survey paper *Credit Card Fraud Detection Using Machine Learning: A Survey* by Lucas and Jurgovsky. This paper explained important fraud detection challenges such as class imbalance, concept drift, and the importance of recall and F1-score for imbalanced datasets. Based on this paper, we focused on SMOTE, PR-AUC, recall, and confusion matrix analysis.

3.2. Exploratory Data Analysis

EDA was performed to understand transaction patterns before model training.

Class Distribution

The class distribution plot shows that the dataset is highly imbalanced. Class 0, which represents legitimate transactions, appears much more frequently than Class 1, which represents fraudulent transactions. This result confirms that accuracy alone is not enough for evaluation, because a model could predict most transactions as legitimate and still achieve high accuracy. This is the main reason why SMOTE and imbalance-focused metrics were used in this project..

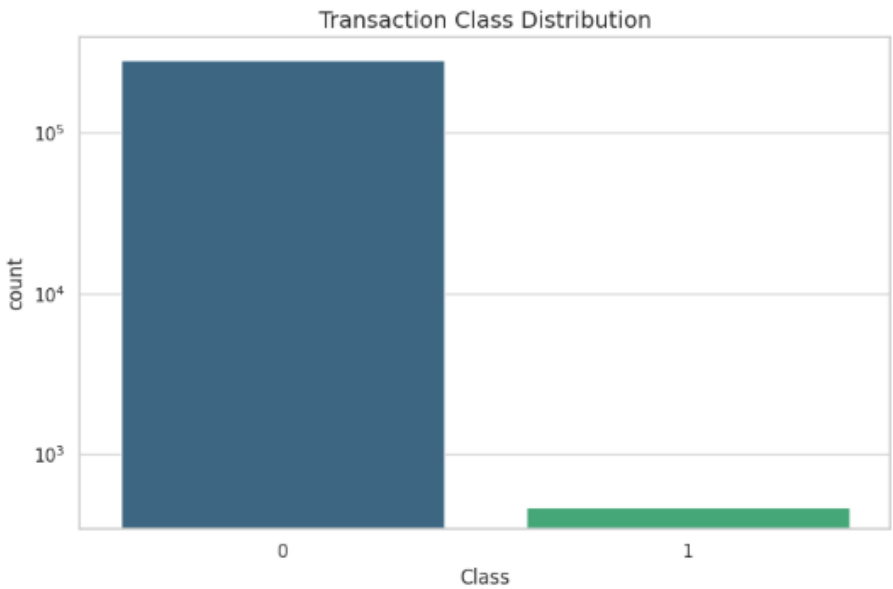


Figure 1: Transaction Class Distribution

Transaction Density by Hour

The transaction density by hour plot compares normal and fraud transaction patterns across time. The plot shows that fraud and legitimate transactions do not have exactly the same hourly distribution. This supports the decision to create the Hour feature

from the Time column. The Hour feature gives the models additional information about time-based transaction behavior, which may help improve fraud prediction.

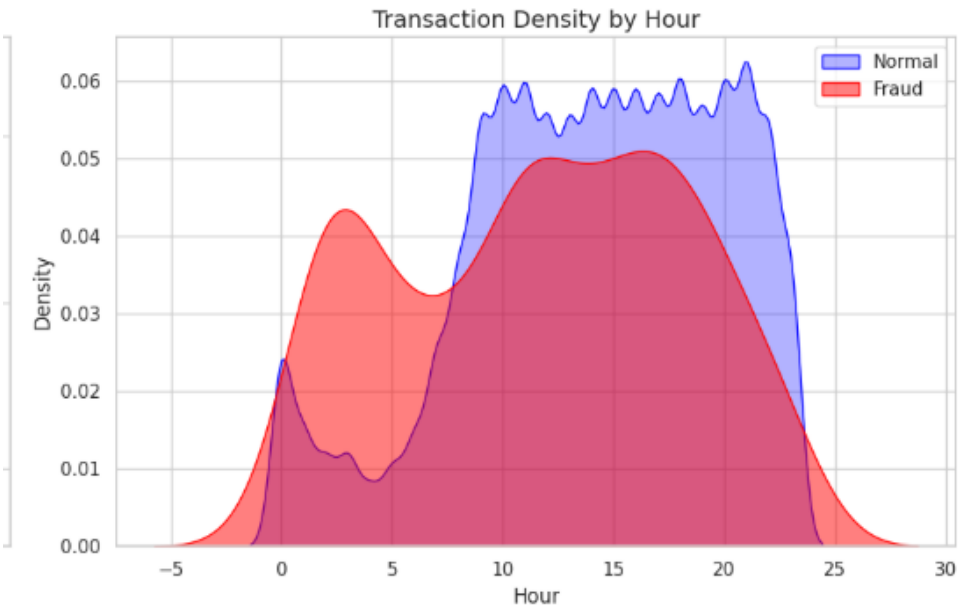


Figure 2: Transaction Density by Hour

Log Amount Distribution

The Log_Amount distribution shows differences between legitimate and fraudulent transaction amounts. The logarithmic transformation reduces the effect of extreme transaction values and makes the distribution easier to compare. This feature helps the model use transaction amount information more effectively without being dominated by very large values.

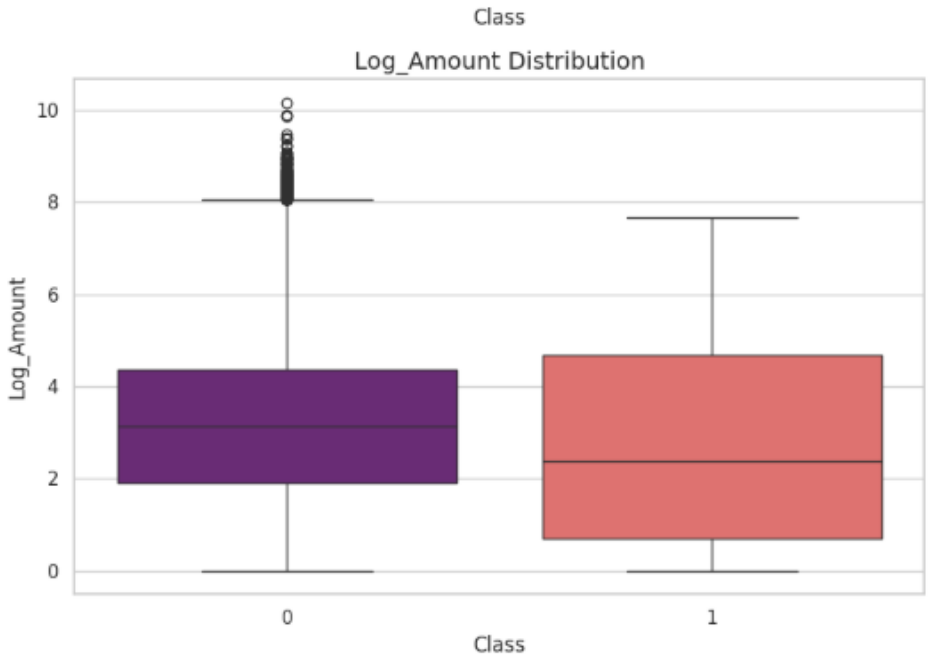


Figure 3: Log Amount Distribution by Class

Correlation Matrix

The correlation matrix shows relationships between numerical features and the target variable. Some anonymized PCA features have stronger relationships with Class than others. This helps explain why tree-based models such as Random Forest and XGBoost can perform well, because they can use combinations of important features to separate fraud and legitimate transactions.

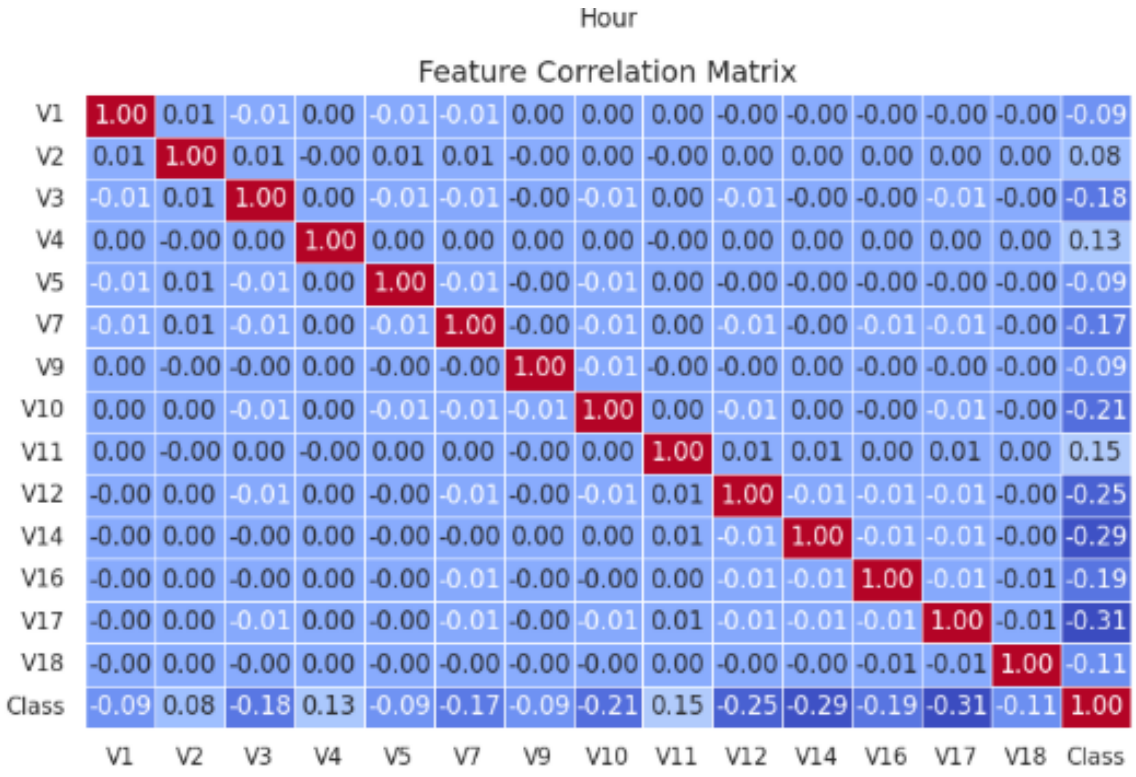


Figure 4: Feature Correlation Matrix

The EDA confirmed that class imbalance is the main challenge in this dataset.

4. Machine Learning Experiments

Three machine learning models were trained and compared:

- Logistic Regression
- Random Forest
- XGBoost

Logistic Regression was used as a baseline model because it is simple and interpretable.

Random Forest was selected because ensemble tree-based models perform strongly on nonlinear classification tasks.

XGBoost was included because boosting methods are powerful for structured tabular datasets and can capture complex transaction patterns effectively.

4.1. Hyperparameter Settings

The models were trained using fixed hyperparameter settings. Logistic Regression was used as a baseline model, while Random Forest and XGBoost were used as stronger ensemble-based models.

Model	Hyperparameters
Logistic Regression	max_iter = 1000, random_state = 42
Random Forest	n_estimators = 150, max_depth = 10, random_state = 42, n_jobs = -1
XGBoost	n_estimators = 150, learning_rate = 0.1, max_depth = 6, subsample = 0.8, colsample_bytree = 0.8, random_state = 42, eval_metric = logloss

Table 2: Hyperparameter settings used for model training

For Random Forest, n_estimators controls the number of trees, while max_depth limits the depth of each tree to reduce overfitting. For XGBoost, learning_rate controls how strongly each tree contributes to the final model, and subsample and colsample_bytree help reduce overfitting by using only part of the data and features during training.

4.2. Evaluation Metrics

The models were evaluated using:

- Precision
- Recall
- F1-score
- ROC-AUC
- PR-AUC
- Confusion Matrix

Precision:

$$Precision = \frac{TP}{TP + FP}$$

Recall:

$$Recall = \frac{TP}{TP + FN}$$

F1-score:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Recall is especially important because false negatives may lead to financial losses.

4.3. Model Performance Comparison

Model	Precision	Recall	F1-score	ROC-AUC	PR-AUC
Logistic Regression	0.1312	0.9667	0.2311	0.9832	0.7221
Random Forest	0.7778	0.9333	0.8485	0.9989	0.9264
XGBoost	0.9655	0.9333	0.9492	0.9987	0.9465

Table 3: Model Performance Comparison

XGBoost achieved the best overall performance with the highest precision, F1-score, and PR-AUC. Random Forest also performed strongly, while Logistic Regression produced many false positives despite high recall.

4.4. Confusion Matrix Analysis

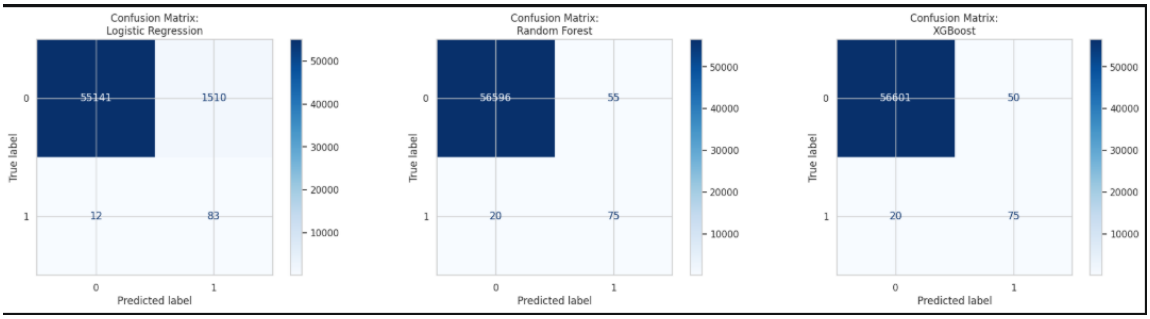


Figure 5: Confusion Matrices for Logistic Regression, Random Forest, and XGBoost

The confusion matrices show how many transactions were correctly and incorrectly classified by each model. Logistic Regression detected most fraud cases, but it produced many false positives, meaning many legitimate transactions were incorrectly classified as fraud. Random Forest reduced the number of false positives while still detecting most fraud cases. XGBoost produced the best balance, with very few false positives and strong fraud detection performance. This result shows that XGBoost is more reliable for practical fraud detection.

4.5. ROC Curve Analysis

The ROC curves compare how well each model separates fraud and legitimate transactions. Random Forest and XGBoost achieved ROC-AUC values close to 1, showing that they can distinguish between the two classes very effectively. Logistic Regression also performed well, but its overall separation ability was slightly lower than the ensemble models.

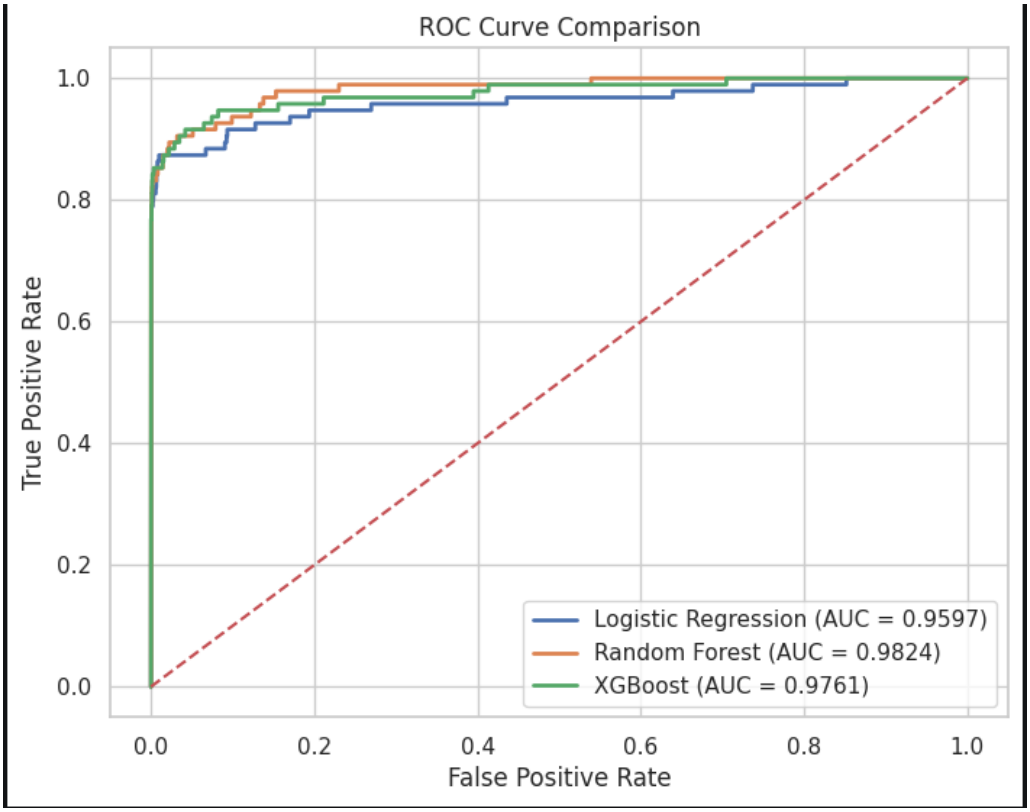


Figure 6: ROC Curve Comparison

4.6. Precision-Recall Curve Analysis

The Precision-Recall curve is especially important for this project because the dataset is imbalanced. XGBoost achieved the highest PR-AUC, meaning it had the best balance between precision and recall. Random Forest also performed strongly, while Logistic Regression had lower precision. This confirms that XGBoost is the strongest model for detecting fraud while avoiding too many false alarms.

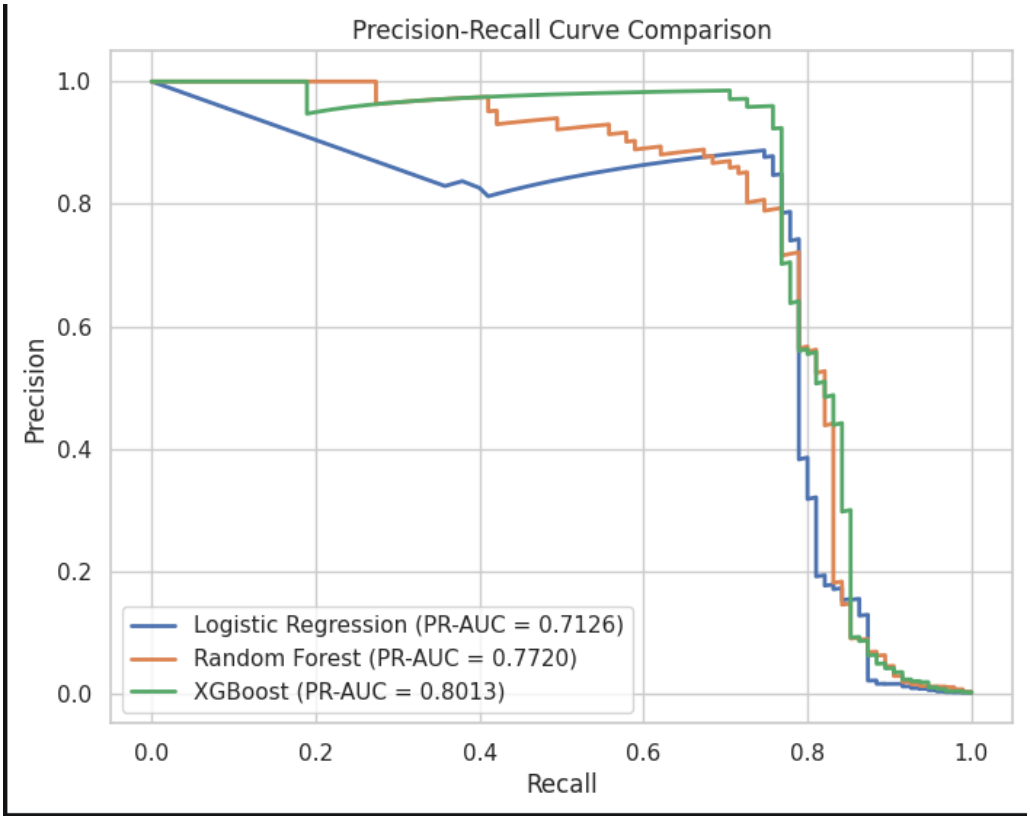


Figure 7: Precision-Recall Curve Comparison

4.7. Feature Importance

The feature importance plot shows which features contributed most to the tree-based models. For both Random Forest and XGBoost, features such as V14 and V10 were among the most important. Since V1–V28 are anonymized PCA features, their real meanings are unknown, but the plot still helps us understand which transformed features were most useful for fraud prediction.

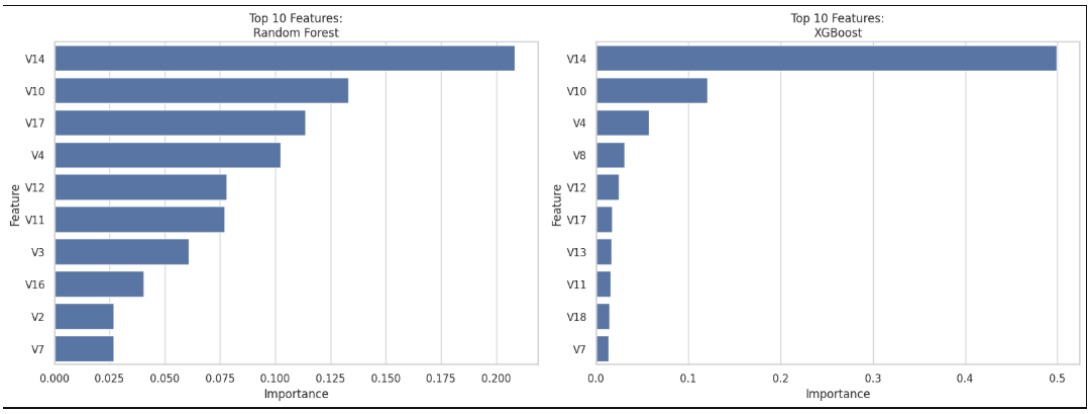


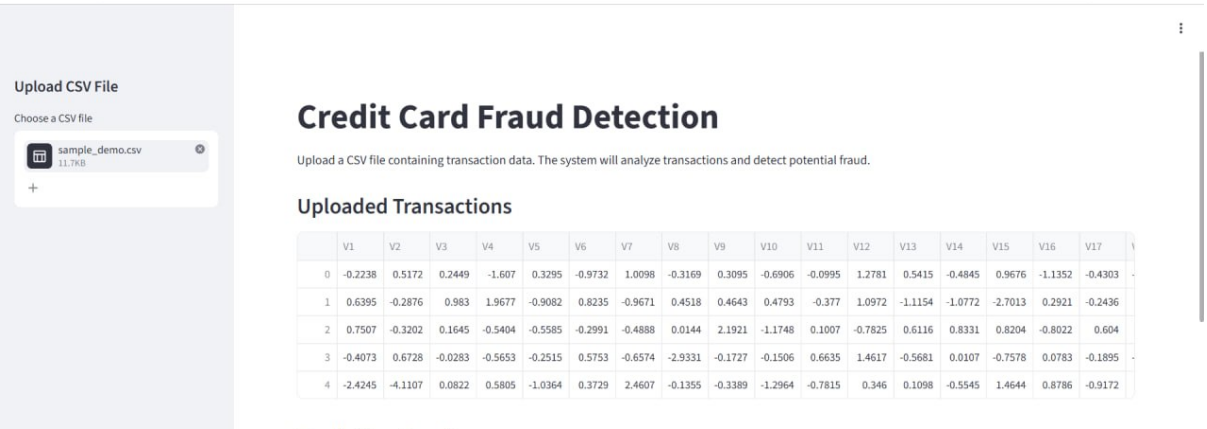
Figure 8: Top Important Features

5. Final Model and Demonstration

The final model was selected based on PR-AUC, F1-score, recall, and precision.

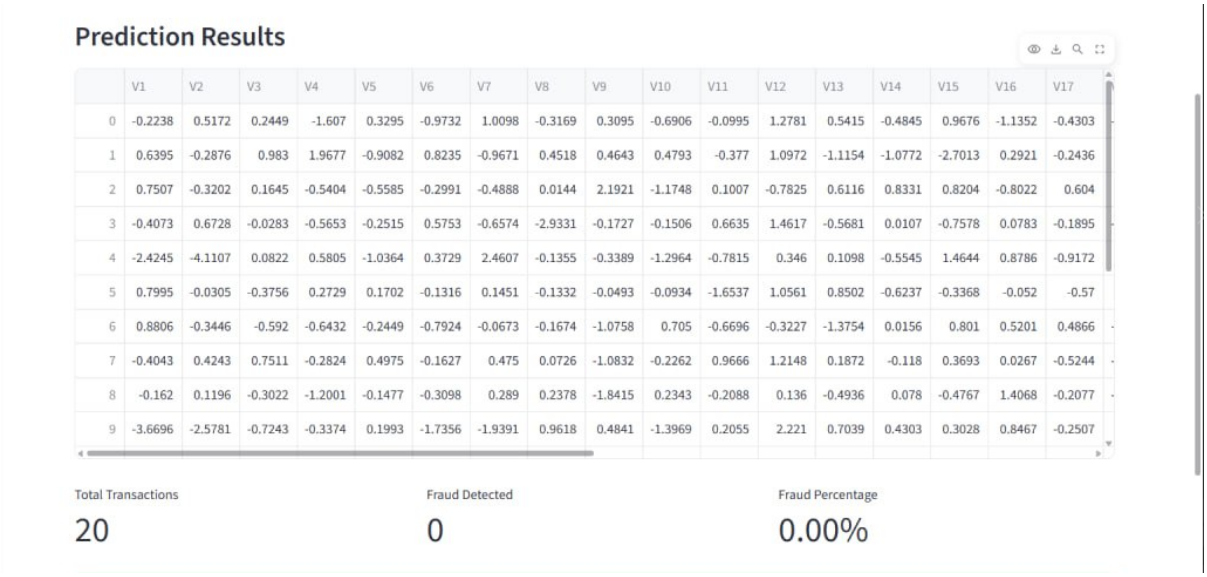
XGBoost achieved the strongest overall fraud detection performance and was selected as the final model.

An interactive fraud detection demonstration was developed to show how the trained machine learning model can classify transactions as fraudulent or legitimate. The application allows users to upload a CSV file containing transaction data. After uploading the file, the system preprocesses the data, applies the trained fraud detection model, and generates prediction results. The demo also displays the total number of transactions, detected fraud cases, and fraud percentage.



	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	V11	V12	V13	V14	V15	V16	V17
0	-0.2238	0.5172	0.2449	-1.607	0.3295	-0.9732	1.0098	-0.3169	0.3095	-0.6906	-0.0995	1.2781	0.5415	-0.4845	0.9676	-1.1352	-0.4303
1	0.6395	-0.2876	0.983	1.9677	-0.9082	0.8235	-0.9671	0.4518	0.4643	0.4793	-0.377	1.0972	-1.1154	-1.0772	-2.7013	0.2921	-0.2436
2	0.7507	-0.3202	0.1645	-0.5404	-0.5585	-0.2991	-0.4888	0.0144	2.1921	-1.1748	0.1007	-0.7825	0.6116	0.8331	0.8204	-0.8022	0.604
3	-0.4073	0.6728	-0.0283	-0.5653	-0.2515	0.5753	-0.6574	-2.9331	-0.1727	-0.1506	0.6635	1.4617	-0.5681	0.0107	-0.7578	0.0783	-0.1895
4	-2.4245	-4.1107	0.0822	0.5805	-1.0364	0.3729	2.4607	-0.1355	-0.3389	-1.2964	-0.7815	0.346	0.1098	-0.5545	1.4644	0.8786	-0.9172

Figure 9: Fraud Detection Demo Interface and Uploaded Transaction Data



	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	V11	V12	V13	V14	V15	V16	V17
0	-0.2238	0.5172	0.2449	-1.607	0.3295	-0.9732	1.0098	-0.3169	0.3095	-0.6906	-0.0995	1.2781	0.5415	-0.4845	0.9676	-1.1352	-0.4303
1	0.6395	-0.2876	0.983	1.9677	-0.9082	0.8235	-0.9671	0.4518	0.4643	0.4793	-0.377	1.0972	-1.1154	-1.0772	-2.7013	0.2921	-0.2436
2	0.7507	-0.3202	0.1645	-0.5404	-0.5585	-0.2991	-0.4888	0.0144	2.1921	-1.1748	0.1007	-0.7825	0.6116	0.8331	0.8204	-0.8022	0.604
3	-0.4073	0.6728	-0.0283	-0.5653	-0.2515	0.5753	-0.6574	-2.9331	-0.1727	-0.1506	0.6635	1.4617	-0.5681	0.0107	-0.7578	0.0783	-0.1895
4	-2.4245	-4.1107	0.0822	0.5805	-1.0364	0.3729	2.4607	-0.1355	-0.3389	-1.2964	-0.7815	0.346	0.1098	-0.5545	1.4644	0.8786	-0.9172
5	0.7995	-0.0305	-0.3756	0.2729	0.1702	-0.1316	0.1451	-0.1332	-0.0493	-0.0934	-1.6537	1.0561	0.8502	-0.6237	-0.3368	-0.052	-0.57
6	0.8806	-0.3446	-0.592	-0.6432	-0.2449	-0.7924	-0.0673	-0.1674	-1.0758	0.705	-0.6696	-0.3227	-1.3754	0.0156	0.801	0.5201	0.4866
7	-0.4043	0.4243	0.7511	-0.2824	0.4975	-0.1627	0.475	0.0726	-1.0832	-0.2262	0.9666	1.2148	0.1872	-0.118	0.3693	0.0267	-0.5244
8	-0.162	0.1196	-0.3022	-1.2001	-0.1477	-0.3098	0.289	0.2378	-1.8415	0.2343	-0.2088	0.136	-0.4936	0.078	-0.4767	1.4068	-0.2077
9	-3.6696	-2.5781	-0.7243	-0.3374	0.1993	-1.7356	-1.9391	0.9618	0.4841	-1.3969	0.2055	2.221	0.7039	0.4303	0.3028	0.8467	-0.2507

Figure 10: Prediction Results Generated by the Fraud Detection Model

6. Analysis and Conclusion

This project developed a complete machine learning pipeline for credit card fraud detection.

The pipeline included data cleaning, feature engineering, scaling, SMOTE balancing, model training, and model evaluation.

The main challenge was severe class imbalance because fraudulent transactions represented only a small fraction of the dataset. To solve this problem, SMOTE was applied to the training data and imbalance-focused evaluation metrics were used.

Three machine learning models were trained and compared: Logistic Regression, Random Forest, and XGBoost. Among these models, XGBoost achieved the best overall performance based on precision, F1-score, and PR-AUC.

Overall, the project demonstrates that machine learning can effectively support fraud detection systems when proper preprocessing, imbalance handling, and evaluation strategies are applied.

6.1. Limitations

This project has several limitations.

First, the dataset contains anonymized PCA-transformed features, making feature interpretation difficult.

Second, SMOTE creates synthetic fraud examples that may not perfectly represent real fraud behavior.

Finally, fraud behavior may change over time, which is known as concept drift.

Future improvements may include behavioral feature engineering, real-time fraud detection systems, and additional ensemble learning approaches.

7. References

1. Afriyie, J. K., Tawiah, K., Pels, W. A., Addai-Henne, S., Dwamena, H. A., Owiredu, E. O., Ayeh, S. A., and Eshun, J. *A Supervised Machine Learning Algorithm for Detecting and Predicting Fraud in Credit Card Transactions.*

2. Kaggle Notebook. *creditcard-fraud-random-forest-example*

3. Lucas, Y., and Jurgovsky, J. *Credit Card Fraud Detection Using Machine Learning: A Survey*, arXiv, 2020.